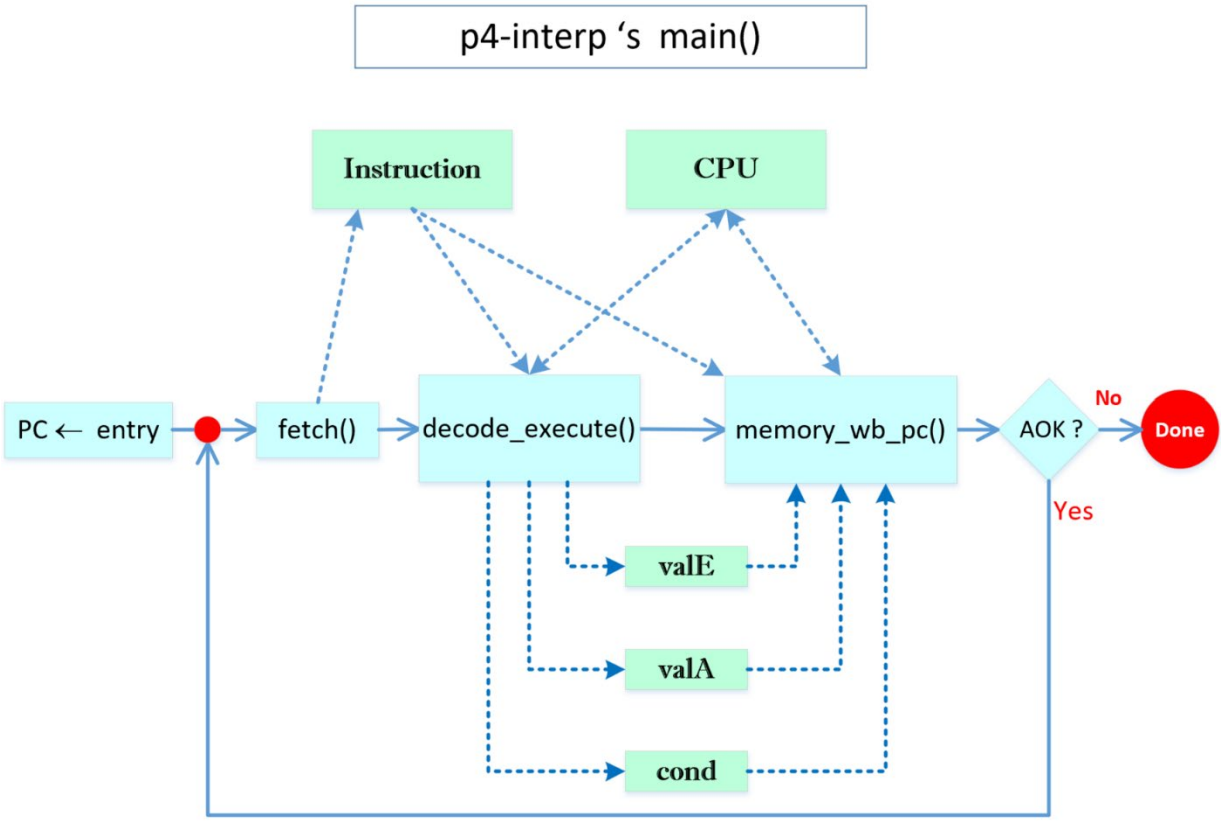


You are free (and encouraged!) to define multiple helper functions, as well. While you are free to call them anything you want, good coding practices recommend you provide meaningful names and clear purposes. All helper functions must be prototyped and implemented inside `p4-interp.c` . **You must never modify `p4-interp.h` or any of the other header files provided to you!** Regardless, the functions above must execute correctly according to specification.

Error checking

Robust software proactively checks for potential errors and responds appropriately when an error occurs. Failure to build robust software leads to security breaches, lost sales, and other problems; this failure is not acceptable. Our grading procedures will try to break your code. The following list is a sample (not complete) of the types of errors we may test:

- Passing NULL values in pointer parameters.
- Passing a file that contains invalid opcodes.
- Passing a file with addresses that would extend beyond the maximum `MEMSIZE` value.
- Passing a file with program headers that have invalid flags or types.
- Passing invalid instructions to the required functions.

In the case of invalid parameters or an invalid instruction, `decode_execute` should set the CPU status code to `INS` if possible, and return a 0. In the case of an invalid memory address, `memory_wb_pc` should set the CPU status code to `ADR`.
The above list is not necessarily exhaustive, and you should think carefully about what sort of errors might occur so that you can prevent them or perform additional error checking as needed. In particular, we will also use **valgrind** to detect memory leaks. Failure to respond appropriately will result in grade reductions.